

Table des matières

Exercices 04 — Fonctions de fenêtrage	1
Questions courtes	1
Questions pratiques	2
Diagnostics	3
Questions à développer	4
Mises en situation	4

Exercices 04 — Fonctions de fenêtrage

Comment travailler ces exercices. Répondez à chaque énoncé seul, puis vérifiez. Les questions courtes vérifient que le vocabulaire est en place ; les pratiques vous font écrire du SQL, tantôt sur le transactionnel, tantôt sur l'entrepôt ; les diagnostics vous montrent du code qui cloche ; les mises en situation vous demandent de choisir une approche et de la défendre. Comptez quatre heures pour y répondre, le corrigé non compris.

Les repères de terrain. Un énoncé marqué [OLTP] se traite dans PostgreSQL sur Adventureworks (schémas sales, production). Un énoncé marqué [OLAP] se traite dans DuckDB sur adventureworksdw.duckdb, ouvert par `USE dbo;`. Sans marque, la question vaut pour les deux.

Le corrigé est dans `Corrige_04.md`. Répondez à l'énoncé en entier avant de l'ouvrir : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. La différence de fond. En une phrase, qu'est-ce qu'une fonction de fenêtre fait que `GROUP BY` ne fait pas ?
2. Les trois morceaux. Que contient un `OVER (...)` au maximum, et à quoi sert chacun des trois éléments ?
3. `PARTITION BY` contre `GROUP BY`. Les deux « regroupent ». Quelle est la différence sur le nombre de lignes en sortie ?
4. Les trois classements. Sur les prix 100, 90, 90, 80, donnez ce que produisent `ROW_NUMBER`, `RANK` et `DENSE_RANK`.
5. Choisir son classement. Vous voulez garder exactement une ligne par groupe, même en cas d'égalité. Laquelle des trois fonctions choisissez-vous, et pourquoi pas les deux autres ?
6. `NTILE`. À quoi sert `NTILE(4)` ? Que se passe-t-il si le nombre de lignes n'est pas divisible par 4 ?

7. LAG et LEAD. Que renvoie LAG(ventes) OVER (ORDER BY mois) sur la toute première ligne, et pourquoi ?
 8. L'argument oublié. LAG(ventes, 3, 0) : que valent le deuxième et le troisième argument ?
 9. La fenêtre vide. Que fait SUM(x) OVER (), sans rien entre les parenthèses ?
 10. Le cadre par défaut. Quand vous écrivez SUM(x) OVER (ORDER BY d), quel cadre s'applique sans que vous l'ayez demandé ? Et quand il n'y a pas d'ORDER BY ?
 11. ROWS contre RANGE. Donnez la différence en une phrase, puis un cas concret où les deux donnent un résultat différent.
 12. Le piège du défaut. Pourquoi une moyenne mobile écrite AVG(x) OVER (ORDER BY d) sans cadre explicite n'est-elle pas une moyenne mobile ?
 13. QUALIFY. À quoi sert-il, et sur quel moteur de cette formation existe-t-il ?
 14. Pourquoi pas le WHERE. Expliquez pourquoi on ne peut pas filtrer sur une fonction de fenêtre directement dans le WHERE. Rattachez votre réponse à l'ordre d'exécution du Cours 01.
 15. L'équivalent portable. Sans QUALIFY, comment obtient-on le même résultat ? Nommez les deux écritures possibles.
 16. [OLAP] La dimension temps. Pour obtenir l'année et le mois d'une vente sur l'entrepôt, que faites-vous ?
 17. [OLTP] La même chose sans dimension. Et sur le transactionnel, où vit la date, et quelle fonction la découpe ?
 18. Le nombre de jointures. Pour une même question analytique, laquelle des deux voies demande généralement le plus de jointures, et pourquoi est-ce voulu de part et d'autre ?
1. Ce qui ne change pas. Nommez quatre éléments de la syntaxe de fenêtrage qui sont rigoureusement identiques sur les deux moteurs de la formation. Nommez ensuite le seul qui diffère.
 2. Le coût caché. Une fonction de fenêtre impose une opération au moteur, laquelle ? Sur PostgreSQL, quel levier permet de la lui épargner, et à quelle condition ce levier sert-il vraiment ? Le même levier vaut-il sur l'entrepôt ?
 3. Fenêtre et agrégat ensemble. Peut-on écrire SUM(SUM(x)) OVER (...) ? Si oui, que fait chaque niveau ?
 4. La part du total. Écrivez, en une ligne et sans contexte de table, l'expression qui donne la part d'une ligne dans la somme de toutes les lignes.

Questions pratiques

Écrivez la requête complète avant de regarder. Quand un énoncé existe en deux versions, faites les deux et comparez-les : c'est là que le contraste entre les deux mondes devient concret.

- P1. [OLAP] Rang par couleur. Pour chaque produit, affichez son nom, sa couleur, son prix et son rang de prix à l'intérieur de sa couleur (le plus cher vaut 1). Ignorez les produits sans couleur et sans prix.
- P2. [OLTP] Rang par couleur, l'autre voie. La même chose sur `production.product`. Notez ce qui change et ce qui ne change pas.
- P3. [OLAP] Champion par sous-catégorie. Avec `QUALIFY`, ne gardez que le produit le plus cher de chaque sous-catégorie.
- P4. [OLTP] Champion par sous-catégorie, sans `QUALIFY`. Le même résultat dans PostgreSQL. Affichez le libellé de la sous-catégorie, pas son numéro, ce qui vous demandera une jointure de plus.
- P5. Part du total. Pour chaque produit dont le prix est supérieur à zéro, affichez son prix et sa part dans la somme de tous les prix de catalogue. Donnez la requête pour votre terrain préféré, puis dites ce qui changerait sur l'autre.
- P6. [OLAP] Croissance mensuelle. Reprenez les ventes mensuelles et ajoutez une colonne de croissance en pourcentage par rapport au mois précédent. Protégez la division.
- P7. [OLTP] Croissance mensuelle, l'autre voie. La même chose, en découpant `orderdate` vous-même.
- P8. Les trois premiers mois de chaque année. Pour chaque année, ne gardez que les trois mois aux ventes les plus fortes, classés. Écrivez la version portable, celle qui marche des deux côtés.
- P9. Le cumul qui repart. Ajoutez aux ventes mensuelles un cumul qui repart à zéro à chaque nouvelle année, à côté du cumul global. Un seul `OVER` par colonne.
- P10. (avancé) L'écart au meilleur. Pour chaque produit, affichez son prix, le prix du produit le plus cher de sa sous-catégorie, et l'écart en pourcentage entre les deux. Une seule passe sur la table, aucune sous-requête d'agrégat.
- P11. (avancé) La médiane par groupe. Pour chaque sous-catégorie, donnez le prix médian de ses produits, en vous servant uniquement de fonctions de fenêtrage. Dites ensuite quelle fonction aurait fait ça en une ligne, et pourquoi l'exercice vous demande l'autre chemin.
- P12. (avancé) Deux cadres dans une requête. Sur les ventes mensuelles, affichez côte à côte la moyenne des trois derniers mois et la moyenne des trois mois suivants. Dites ce que chaque colonne vaut sur les lignes du bord, combien de lignes sont touchées de chaque côté, et pourquoi.

Diagnostics

On vous montre du code qui ne fait pas ce que son auteur croit. Dites ce qui cloche, pourquoi, et donnez le correctif.

E1. Une analyste veut la moyenne mobile des ventes sur trois mois. Elle écrit :

```
SELECT annee, mois, ventes,  
       AVG(ventes) OVER (ORDER BY annee, mois) AS moy_mobile_3  
FROM mensuel;
```

Le résultat monte doucement et ne ressemble à aucune moyenne mobile. Expliquez ce que la requête calcule réellement, nommez le mécanisme en cause, et corrigez.

E2. Un développeur veut les trois produits les plus chers par sous-catégorie sur PostgreSQL. Il écrit :

```
SELECT name, listprice,  
       ROW_NUMBER() OVER (PARTITION BY productsubcategoryId  
                          ORDER BY listprice DESC) AS rang  
FROM production.product  
WHERE rang <= 3;
```

Le moteur répond que la colonne rang n'existe pas. Il conclut que PostgreSQL ne gère pas les fonctions de fenêtrage. Expliquez-lui son erreur, dites pourquoi le message est trompeur, et donnez les deux écritures correctes.

E3. Une requête calcule la part de chaque vente dans le total annuel :

```
SELECT annee, mois, ventes,  
       ventes / SUM(ventes) OVER (ORDER BY annee) AS part_annee  
FROM mensuel;
```

Les parts de la première année s'additionnent bien à 1, mais celles des années suivantes s'additionnent à de moins en moins : 0,6846, puis 0,4861, puis 0,1821. Et tous les mois d'une même année sont divisés par le même nombre, ce qui ressemble à ce qu'on voulait sans l'être. Trouvez les deux fautes distinctes, et dites pourquoi la première année, elle, tombe juste.

Questions à développer

Un court paragraphe de raisonnement, pas un mot juste.

A. La fenêtre et le grain. Expliquez pourquoi une fonction de fenêtre ne change jamais le grain du résultat, alors qu'un GROUP BY le change toujours. Puis dites en quoi cette propriété est précisément ce qui rend le fenêtrage indispensable au reporting, avec un exemple où l'agrégation classique obligerait à écrire deux requêtes et une jointure.

B. Le même langage, deux terrains. Vous avez fait le laboratoire des deux côtés. Expliquez, à quelqu'un qui n'a vu que l'entrepôt, ce qui l'attend s'il doit écrire la même analyse sur une base transactionnelle. Distinguez ce qui relève de la syntaxe, ce qui relève du schéma, et ce qui relève de la performance.

C. QUALIFY, confort ou piège. QUALIFY rend le code plus court et plus lisible, et il tourne sur l'entrepôt de cette formation. Défendez d'abord son usage, puis donnez l'argument sérieux qu'on peut lui opposer dans une équipe qui travaille sur plusieurs moteurs. Expliquez au passage pourquoi la ligne de partage n'est pas celle qu'on croit, entre gros moteur et petit moteur. Concluez par la règle que vous adopteriez.

Mises en situation

On décrit une entreprise et une question métier. À vous de choisir l'approche, de l'écrire, et surtout de la justifier. Il y a plusieurs bonnes réponses, mais il y a des réponses indéfendables.

M1. Épicerie Bellefeuille, le palmarès qui bouge. Une chaîne de douze épiceries veut un écran hebdomadaire qui montre, pour chaque succursale, ses cinq produits les plus vendus de la semaine, avec leur rang, leur part dans le chiffre de la succursale, et leur progression de rang par rapport à la semaine précédente. Les données vivent dans le système de caisse, une base transactionnelle PostgreSQL, avec une table de tickets et une table de lignes de ticket.

Décrivez la requête que vous écririez : quelles fenêtres, sur quelles partitions, dans quel ordre, et combien de passes sur les données. La progression de rang est le point difficile : dites pourquoi, et comment vous la traitez. Puis répondez à la question qui décide de tout : cet écran doit-il être servi par la base transactionnelle ou par un entrepôt, et qu'est-ce qui tranche ?

M2. Clinique vétérinaire Deux-Montagnes, l'intervalle entre deux visites. La clinique veut savoir, pour chaque animal, le délai entre deux consultations consécutives, afin de repérer les patients qu'on ne revoit plus. Elle veut aussi le délai moyen par espèce, et la liste des animaux dont la dernière visite remonte à plus du double de leur délai habituel.

Écrivez le raisonnement complet : quelles fonctions de fenêtrage, dans quel ordre, et combien de niveaux de requête. Précisez ce que vous faites de la toute première visite de chaque animal, qui n'a pas de précédente, et de ceux qui n'ont jamais eu qu'une seule visite. Terminez en disant pourquoi la dernière demande, celle du double du délai habituel, ne peut pas se traiter dans la même fenêtre que les deux premières.

M3. (avancé) Coopérative forestière Abitibi, le contraste imposé. La coopérative a deux systèmes. Le système d'exploitation, transactionnel, contient les livraisons de bois au jour le jour depuis quatre ans, soit 40 millions de lignes. L'entrepôt, alimenté chaque nuit, contient les mêmes livraisons modélisées en étoile. On vous demande le même rapport dans les deux systèmes : la part de chaque coupe dans le volume mensuel de sa région, avec un cumul annuel glissant.

Écrivez les deux requêtes, ou décrivez-les précisément si vous préférez. Puis comparez-les sur quatre axes : le nombre de jointures, la lisibilité, la fraîcheur du résultat, et le risque pour le système. Dites laquelle vous mettriez en production et dans quelles conditions vous changeriez d'avis. Un des quatre axes joue contre l'entrepôt : identifiez-le et ne l'escamotez pas.